

Верификация инструкций динамического разрешения в статически типизированной виртуальной машине.

М. А. Грошев^{1,2}

¹Московский физико-технический институт (национальный исследовательский университет)

² «Российский исследовательский институт»

В настоящее время большой популярностью пользуются языки программирования со статической типизацией. При этом зачастую, для повышения выразительности языка, возникает потребность в динамических типах, а для высокой производительности необходима их поддержка на уровне среды исполнения. Наличие динамики не должно приводить к снижению типовой безопасности или к появлению дополнительных ошибок времени выполнения. Существующие платформы, такие как Android Runtime (ART) и Java Virtual Machine (JVM), изначально ориентированы преимущественно на строго статическую модель типов и не предоставляют специализированного набора инструкций и метаданных, сохраняемой до момента исполнения, напрямую предназначенной для эффективной и безопасной работы с динамически разрешаемыми типами. Разработка алгоритмов верификации подобных инструкций является целью данной работы.

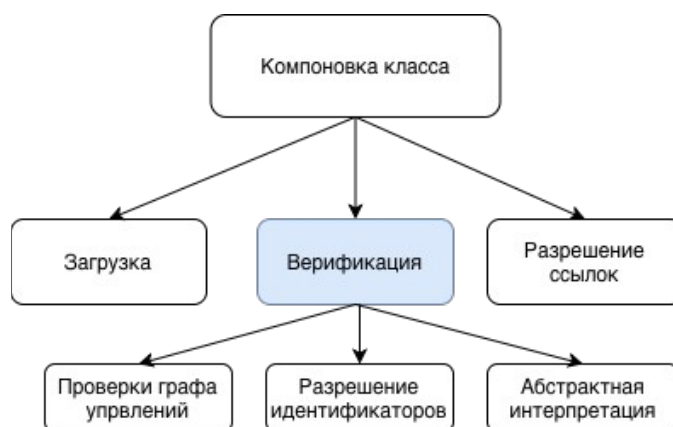


Рис. 1. Диаграмма, демонстрирующая процесс компоновки и верификации класса.

Целевая платформа в рамках своей системы типов предусматривает поддержку типов, разрешаемых на этапе выполнения программы. В отличие от статической модели, такая система допускает существование типов, конкретное значение которых может изменяться в процессе исполнения, что требует специальных механизмов анализа и контроля со стороны верификатора.

К числу таких типов относится `union`-тип — составной тип данных, представляющий объединение нескольких заранее определённых вариантов. Объект, имеющий `union`-тип, может во время выполнения принимать значение любого из входящих в объединение типов, при этом корректность операций над ним должна подтверждаться с учётом всех возможных вариантов. Это существенно усложняет задачу статической проверки, поскольку верификатору необходимо учитывать множество потенциальных ветвей типового состояния.

Кроме того, система типов в себе имеет `any`-тип, который допускает присвоение объекту значения произвольного типа, включая те, которые определяются на стороне динамически типизированного языка программирования (контекст интеропы). Использование `any`-типа предоставляет максимальную гибкость при взаимодействии с динамически типизированными компонентами, однако одновременно создаёт дополнительные требования к механизмам верификации.

В ходе работы, в систему верификаторных типов были интегрированы union-типы и разработаны алгоритмы верификации инструкций имеющих динамический характер. Это позволило повысить безопасность платформы и предотвратить ряд ошибок времени выполнения ещё на этапе верификации при установке.

Таким образом, предложенные решения позволили поддержать инструкции, по своей сути являющиеся динамическими, обеспечив при этом сохранение возможностей статической верификации, что является ключевым преимуществом статически типизированных виртуальных машин.

Литература

1. ArkTs Documentation [Эл. ресурс]. URL: https://gitcode.com/igelhaus/arkcompiler_runtime_core/releases (дата обращения: 23.02.2026).
2. Ecma Documentation [Эл. ресурс]. URL: <https://tc39.es/ecma262> (дата обращения: 23.02.2026)
3. JVM Specification [Эл. ресурс]. URL: <https://docs.oracle.com/javase/specs/jvms/se8/html> (дата обращения: 23.02.2026)